

Power Platform Best Practices

Version 1.0

Version Date: Jun 15, 2026

Created by:

- Ricardo de Souza

Contributors

Overview	2
Architecture & Solution Design Principles	2
Development Best Practices	3
Performance & Scalability.....	4
Security & Compliance Best Practices	5
ALM & DevOps Practices	5
Monitoring & Maintenance	6
Cause Catalog for the Performance Issues.....	7
C01 — Non-delegable query on large dataset.....	7
C02 — Excessive data loading at app start	8
C03 — Large dataset without data shaping or pagination	10
C04 — Monolithic app or flow design	11
C05 — Incorrect tool/component used for the workload	12
C06 — Long-running synchronous process instead of asynchronous design.....	14
C07 — Inefficient query/filter design in flows or data access	15
C08 — Inefficient Power Query/dataflow transformations	16
C09 — Misused parallelism or unsafe concurrency.....	17
C10 — Lack of scale testing before production	18
C11 — Poor reuse/component strategy leading to repeated heavy logic	20
C12 — Missing or weak monitoring for degraded performance.....	21
C13 — Datastore choice not aligned with solution needs	22
Performance Remediation Primitives.....	23
List of Remediation Primitives	24
Common Primitive Combinations.....	30
Official Microsoft Learn References	31
Conclusion	31

Overview

This document summarizes **best practices across architecture, solution design, development, performance, security, and operations** for Power Platform projects. Applying these practices helps ensure solutions are **scalable, maintainable, and aligned with Microsoft's Well-Architected Framework** pillars: Reliability, Security, Performance, Operational Excellence, and User Experience. Each section highlights key principles and recommendations for solution architects and makers.

Architecture & Solution Design Principles

High-level architectural decisions have a significant impact on solution success. Consider these principles during planning and design:

- **Design for High Reliability:** Architect apps and flows to tolerate failures and minimize downtime.
 - Prefer an asynchronous design for long-running processes, such as having a flow queue work via Dataverse rather than holding a single run for hours.
 - Implement error handling and retry policies in flows to recover from transient issues without manual intervention.
 - Use **multiple environments** such as Dev, Test, and Prod to separate concerns and ensure reliability through proper testing before production.
- **Simplicity & Modularity:** Keep designs as simple as possible. Break down complex solutions into modular components.
- Use multiple smaller apps or flows rather than one large, monolithic component, when appropriate. For example, if one flow performs too many unrelated tasks, consider splitting it logically.
- Avoid over-engineering. If a requirement can be met with configuration instead of custom code, use the simpler approach.
- **Use the Right Tool for the Job:** Ensure each Power Platform component is used in an optimal way.
 - Avoid using a Canvas App as a scheduled background processor or an automated flow to present data interactively.
 - Leverage **Dataverse** as the central data store for relational or mission-critical data rather than spreadsheets or lists, when applicable.

- **User Experience Optimization:** Design with the end user in mind to maximize adoption and satisfaction.
 - For Canvas apps, follow UI/UX best practices, including intuitive navigation, responsive layouts, and accessibility.
 - For Power Pages, ensure cross-browser compatibility and fast load times by optimizing images and enabling CDN for static content.

Development Best Practices

During solution implementation, whether building apps, flows, or pages, follow development and configuration standards that improve quality:

- **Naming Conventions:** Adopt a clear naming convention for all artifacts, including screens, controls, actions, variables, tables, and columns. *For example, prefix Dataverse tables with a publisher prefix such as abc_ and flows with their purpose, such as OrderProcessing_SendEmail.*
- **Source Control & ALM:** Store unmanaged solution source in source control when the project requires team development, traceability, or repeatable releases. Use Power Platform CLI, Power Platform Build Tools, GitHub Actions, or native Git integration where appropriate. Treat managed solutions as deployment artifacts for test, UAT, and production environments.
- **Error Handling & Testing:** Build apps and flows defensively.
- In Canvas apps, use functions like `IsBlank()`, `IsError()`, and **IfError** to capture and handle runtime issues.
- In Power Automate flows, implement **try-catch-finally patterns** using Scope actions for robust error handling.
- Perform thorough **unit testing** for each component, including flow tests, canvas app formula tests, and user acceptance testing.
- **Reuse and Components:** Avoid repetition by leveraging reusable assets.
- Use **Canvas app components** through a component library for UI elements that appear across screens or apps.
- Use **child flows** in Power Automate for common sequences that multiple flows may need.
- If building multiple similar apps, consider developing a reference template that can be reused to ensure consistency.

- **Documentation & Comments:** Keep the solution maintainable by documenting how things work.
 - Comment complex formulas or flow sections with short notes that explain intent.
 - Maintain an **architecture diagram** or at least a solution overview that shows the involved components and data flows.

Performance & Scalability

Design and implement solutions to perform well as data volumes and user demand grow:

- **Delegation & Data Shaping:** In Canvas apps, always consider **delegation** when accessing large data sets. Use delegable operations so filtering and sorting happen at the data source. If delegation is not possible, limit data requests or use techniques such as **pagination**.
- **Minimize On-Load Work:** Don't load more data or carry out more logic than necessary at startup.
 - For Canvas apps, load only what the user needs initially, then lazy-load additional data on demand.
 - For flows, use targeted checks and filter queries to fetch only relevant records.
- **Optimize Power Query and Dataflows:** When using dataflows for ETL into Dataverse or a data warehouse, filter data at the source and avoid unnecessary transformation steps. Set incremental refresh if supported.
- **Use Parallelism Thoughtfully:** Power Automate allows parallel branches and concurrency in loops. Use these features to reduce overall run time for independent actions, but be cautious with race conditions and conflicting updates.
- **Testing at Scale:** Anticipate higher data volumes and user loads. If possible, simulate load testing with large data sets in a test environment and use performance insights to detect bottlenecks.

Security & Compliance Best Practices

Security should be integrated from the start:

- **Least Privilege Access:** Grant users the minimal necessary privileges.
 - For Dataverse, design security roles so users get row-level and column-level access only as needed.

- For Power Apps, remember that Canvas apps run with the end user's permissions. Ensure the data source itself has controlled access.
- **Environment Separation for Data Sensitivity:** Keep sensitive, business-critical, or regulated solutions in dedicated environments with appropriate security groups, admin ownership, and lifecycle controls. Use **data loss prevention (DLP) policies** to govern connector combinations and reduce the risk of unintended data movement.
- **Data Encryption:** Dataverse data is automatically encrypted at rest by Microsoft. For extremely sensitive data, consider **customer-managed keys** for additional control over encryption keys, where applicable.
- **Compliance Features:** If applicable, enable **field-level auditing** and **data retention policies** in Dataverse to meet compliance guidelines.
- **Secure DevOps:** For automated deployments, use service principals or approved service accounts with least-privilege access. Store credentials and secrets in secure stores such as Azure Key Vault, avoid hard-coding secrets in pipelines or flows, and review access regularly.

ALM & DevOps Practices

Adopt application lifecycle management (ALM) practices that make releases predictable, auditable, and reversible where possible:

- **Solutions for Everything:** Build apps, cloud flows, tables, security roles, connection references, environment variables, and other ALM-aware components inside solutions from the start. Avoid building production assets directly in the default solution.
- **Managed and Unmanaged Solutions:** Use unmanaged solutions for active development and source control. Export managed solutions as release artifacts for downstream environments such as test, UAT, and production. Avoid direct customization in production because it creates unmanaged layers and can complicate upgrades and support.
- **Automate Build & Deployment:** Use **Power Platform Pipelines** for governed in-product deployments, or use Azure DevOps and GitHub Actions when more advanced CI/CD, branching, approvals, or enterprise DevOps integration is required. Automated deployment reduces manual error and improves release traceability.
- **Solution Checker & Reviews:** Run **Solution Checker** regularly as part of quality gates to identify performance, reliability, maintainability, and security issues. Combine

automated checks with peer reviews for complex apps, flows, custom connectors, plug-ins, and critical business processes.

- **Deployment Settings:** Use connection references and environment variables to separate configuration from implementation. This makes deployments more reliable across environments and reduces the need for manual changes after import.

Monitoring & Maintenance

Support reliable operations through active monitoring and a clear maintenance plan:

- **Establish Monitoring:** For critical processes, set up notifications and operational review routines for failures, exceptions, and degraded performance. Use Power Platform admin center experiences, platform analytics, and available telemetry integrations to monitor apps, flows, agents, and environment health.
- **Governance Operations:** Use Power Platform admin center capabilities such as Inventory, Usage, Monitor, and Actions as the primary governance experience. Treat the CoE Starter Kit as a reference implementation or legacy deployment option because Microsoft indicates that its core scenarios are moving into the admin center and the kit is no longer actively maintained.
- **Review and Optimize Regularly:** Periodically conduct a **Well-Architected Review** to identify drift from best practices as the solution evolves.
- **Backup and Disaster Recovery:** Ensure backups of key environments are available at appropriate frequencies. Document a **disaster recovery plan** for restoring critical functionality if an environment becomes unavailable.
- **Stay Current:** Keep track of Power Platform release plans and Microsoft 365 Message Center announcements for new features, deprecations, and changes that may affect solutions.

Cause Catalog for the Performance Issues

This catalog provides a structured reference for diagnosing common performance issues in Power Platform solutions. Each cause describes the likely scenario, observable symptoms, validation signals, recommended actions, default likelihood and impact, and related correlation candidates. Use it as a practical triage guide to move from user-reported slowness or operational degradation toward evidence-based root cause analysis and targeted remediation.

C01 — Non-delegable query on large dataset

- **Category:** Data / Canvas App
- **Description:** The app is using non-delegable filtering, sorting, or formula patterns against a large data source, causing client-side processing and incomplete or slow results. This is directly derived from the KB guidance to always consider delegation for large datasets and to use delegable operations whenever possible. [[Power Plat...Practices | Word](#)]
- **Typical symptoms:** Slow gallery loads, partial result sets, degraded search/filter responsiveness on large lists or tables.
- **Validation signals:** Delegation warnings in formulas; issue worsens when data volume grows; performance improves when query scope is reduced.
- **Recommended actions:** Refactor formulas to delegable patterns; reduce data scope; use pagination or alternative data access strategies where delegation is not possible. The KB explicitly recommends delegable operations and limiting requests when delegation cannot be achieved. [[Power Plat...Practices | Word](#)]

- **Default likelihood:** High
- **Default impact:** High
- **Correlation candidates:** Large dataset, excessive on-load work, complex formulas
- **KB reference:** *Performance & Scalability → Delegation & Data Shaping*

C02 — Excessive data loading at app start

- **Category:** Data Loading / Canvas App
- **Description:** The app loads more data or executes more logic than necessary during initialization, increasing startup time and perceived slowness. This is derived from the KB recommendation to minimize on-load work and load only what the user needs initially. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Slow app startup, delayed first interaction, long splash/loading states.
- **Validation signals:** Large collections loaded in OnStart; many connector calls before first screen is usable; better performance when startup logic is disabled or deferred.
- **Recommended actions:** Lazy-load data, defer non-critical operations, fetch only what is required for the initial user task. The KB explicitly recommends loading only necessary initial data and using on-demand loading. [\[Power Plat...Practices | Word\]](#)
- **Default likelihood:** High
- **Default impact:** High
- **Correlation candidates:** Non-delegable queries, large dataset, connector latency
- **KB reference:** *Performance & Scalability → Minimize On-Load Work*

C03 — Large dataset without data shaping or pagination

- **Category:** Data / Scalability
- **Description:** The solution attempts to process or display too much data at once without narrowing scope, filtering at source, or paginating results. This is derived from the KB's emphasis on delegation, filtering at source, targeted checks, and pagination techniques. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Performance degrades sharply as data grows; list/grid rendering becomes slow; search and sort operations become unstable.
- **Validation signals:** Queries return broad datasets; no paging strategy; issue becomes severe only at higher volumes.
- **Recommended actions:** Filter earlier, paginate, reduce row retrieval scope, shape data before bringing it into the app or flow. The KB explicitly recommends source-side filtering, targeted queries, and pagination where needed. [\[Power Plat...Practices | Word\]](#)
- **Default likelihood:** High
- **Default impact:** High
- **Correlation candidates:** Non-delegable queries, startup overload, poor architecture decisions
- **KB reference:** *Performance & Scalability → Delegation & Data Shaping / Minimize On-Load Work*

C04 — Monolithic app or flow design

- **Category:** Architecture
- **Description:** A single app or flow is handling too many unrelated responsibilities, creating maintenance and runtime overhead. This is derived from the KB guidance to favor simplicity, modularity, and multiple smaller components instead of one large monolithic component. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Slow overall responsiveness, difficult troubleshooting, changes in one area affect unrelated operations, oversized flows or apps.
- **Validation signals:** One app contains too many screens/components/data sources; one flow performs unrelated tasks end-to-end; performance improves when logic is split.
- **Recommended actions:** Split into modular apps/flows, isolate responsibilities, move reusable logic into child flows or reusable components. The KB explicitly recommends smaller modular components and child flows for common sequences. [\[Power Plat...Practices | Word\]](#)
- **Default likelihood:** Medium
- **Default impact:** High
- **Correlation candidates:** Excessive startup load, poor reuse strategy, weak error handling
- **KB reference:** *Architecture & Solution Design Principles / Development Best Practices*

C05 — Incorrect tool/component used for the workload

- **Category:** Architecture / Solution Fit
- **Description:** The solution uses a Power Platform component for a scenario it is not optimized for, such as using a Canvas app as a background processor or a flow for interactive UI behavior. The KB explicitly warns against using components outside their optimal role. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Persistent slowness despite optimization attempts, fragile behavior, poor scalability for the workload type.
- **Validation signals:** Workload pattern does not match platform component design; issue is structural rather than formula-specific.
- **Recommended actions:** Reassign workload to the appropriate component pattern, redesign for correct platform usage, move long-running processes to asynchronous patterns where appropriate. The KB specifically recommends using the right tool for the job and asynchronous designs for long-running work. [\[Power Plat...Practices | Word\]](#)
- **Default likelihood:** Medium
- **Default impact:** High
- **Correlation candidates:** Monolithic design, long-running flow, connector-heavy process
- **KB reference:** *Architecture & Solution Design Principles*

C06 — Long-running synchronous process instead of asynchronous design

- **Category:** Flow / Architecture
- **Description:** A flow or process keeps a long-running execution path active instead of queueing or decoupling the work asynchronously. The KB explicitly recommends asynchronous designs for long-running processes, for example queueing work through Dataverse instead of holding a single run for hours. [[Power Plat...Practices | Word](#)]
- **Typical symptoms:** Long flow runtimes, unstable executions, timeouts, degraded responsiveness for downstream operations.
- **Validation signals:** Single run handles all processing in one execution path; heavy sequential actions; performance improves when work is split or queued.
- **Recommended actions:** Queue work, split into separate stages, use asynchronous orchestration, reduce time spent in a single run. This is directly aligned with the KB's asynchronous design guidance. [[Power Plat...Practices | Word](#)]
- **Default likelihood:** Medium
- **Default impact:** High

- **Correlation candidates:** Monolithic flow design, poor parallelism strategy, large data processing
- **KB reference:** *Architecture & Solution Design Principles*

C07 — Inefficient query/filter design in flows or data access

- **Category:** Flow / Data Access
- **Description:** The solution retrieves more records than necessary or performs broad checks rather than targeted queries. This comes from the KB guidance for flows to use targeted checks and filter queries to fetch only relevant records. [[Power Plat...Practices | Word](#)]
- **Typical symptoms:** Slow flow executions, excessive API calls, delays before data-dependent steps execute.
- **Validation signals:** Broad “list all” patterns, post-filtering after retrieval, unnecessary loops over irrelevant records.
- **Recommended actions:** Use filter queries, narrow retrieval criteria, reduce record scope, only fetch what is needed. The KB explicitly recommends targeted checks and filter queries. [[Power Plat...Practices | Word](#)]
- **Default likelihood:** High
- **Default impact:** Medium
- **Correlation candidates:** Large dataset, connector latency, poor startup load

- **KB reference:** *Performance & Scalability → Minimize On-Load Work*

C08 — Inefficient Power Query/dataflow transformations

- **Category:** Dataflow / ETL
- **Description:** Dataflows apply unnecessary transformations or fail to filter early at the source, increasing processing cost and refresh time. This is directly derived from the KB guidance to filter at the source, avoid unnecessary transformation steps, and use incremental refresh when supported. [[Power Plat...Practices | Word](#)]
- **Typical symptoms:** Slow refreshes, delayed data availability, ETL bottlenecks affecting downstream app performance.
- **Validation signals:** Many transformation steps, full refresh on large data, no source-side data reduction.
- **Recommended actions:** Reduce transformation steps, filter at the source, use incremental refresh where supported. The KB states these practices explicitly. [[Power Plat...Practices | Word](#)]
- **Default likelihood:** Medium
- **Default impact:** High
- **Correlation candidates:** Large dataset, poor monitoring, architecture misfit

- **KB reference:** *Performance & Scalability → Optimize Power Query and Dataflows*

C09 — Misused parallelism or unsafe concurrency

- **Category:** Flow / Concurrency
- **Description:** Parallel branches or loop concurrency are used without considering race conditions, conflicting updates, or actual workload suitability. The KB recommends using parallelism thoughtfully and warns about race conditions and conflicting updates. [[Power Plat...Practices](#) | [Word](#)]
- **Typical symptoms:** Intermittent failures, inconsistent processing times, data conflicts, hard-to-reproduce slowdowns.
- **Validation signals:** Concurrency enabled in loops; conflicting record updates; performance becomes unstable rather than simply slow.
- **Recommended actions:** Reassess concurrency settings, isolate independent actions, serialize conflicting updates, apply parallelism only where it reduces total runtime safely. This is fully aligned with the KB. [[Power Plat...Practices](#) | [Word](#)]
- **Default likelihood:** Low
- **Default impact:** Medium

- **Correlation candidates:** Long-running flow, monolithic design, connector throttling patterns
- **KB reference:** *Performance & Scalability → Use Parallelism Thoughtfully*

C10 — Lack of scale testing before production

- **Category:** Validation / Performance Engineering
- **Description:** The solution was not tested with realistic data volumes or user load, so bottlenecks only appear in production-like conditions. The KB explicitly recommends testing at scale and simulating load with large datasets in test environments. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Works well in dev/test with small samples but slows down significantly in production.
- **Validation signals:** No documented volume tests; issue appears only with real data size or real concurrency.
- **Recommended actions:** Perform scale tests, reproduce with larger datasets, use performance insights to detect bottlenecks. This is directly drawn from the KB. [\[Power Plat...Practices | Word\]](#)
- **Default likelihood:** High

- **Default impact:** Medium
- **Correlation candidates:** Any data-volume-sensitive cause
- **KB reference:** *Performance & Scalability* → *Testing at Scale*

C11 — Poor reuse/component strategy leading to repeated heavy logic

- **Category:** Development / Maintainability
- **Description:** The solution repeats formulas, UI logic, or flow sequences instead of reusing components or child flows, increasing maintenance complexity and possibly execution overhead. The KB recommends using component libraries, child flows, and reusable templates. [[Power Plat...Practices | Word](#)]
- **Typical symptoms:** Similar logic repeated across screens or flows; difficult optimization because the same problem exists in multiple places.
- **Validation signals:** Duplicate formulas/actions detected; multiple similar flows with slight variations; no reusable component strategy.
- **Recommended actions:** Centralize reusable logic into components, child flows, or templates. This is directly aligned with KB guidance. [[Power Plat...Practices | Word](#)]
- **Default likelihood:** Medium

- **Default impact:** Medium
- **Correlation candidates:** Monolithic design, startup overload, maintainability issues
- **KB reference:** *Development Best Practices → Reuse and Components*

C12 — Missing or weak monitoring for degraded performance

- **Category:** Operations / Monitoring
- **Description:** The environment lacks monitoring, notifications, operational reviews, or analytics, delaying the detection and validation of performance issues. The KB explicitly recommends establishing monitoring, using admin center experiences, platform analytics, and telemetry integrations. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Slow performance is reported informally by users; regression is not detected early; fixes cannot be measured objectively.
- **Validation signals:** No alerts, no routine reviews, no before/after metrics, no use of platform monitoring capabilities.
- **Recommended actions:** Set notifications, use platform analytics/admin center monitoring experiences, establish review routines, define success criteria for

performance validation. This is directly supported by the KB. [\[Power Plat...Practices | Word\]](#)

- **Default likelihood:** High
- **Default impact:** Medium
- **Correlation candidates:** All other causes, because weak monitoring makes them harder to detect and validate
- **KB reference:** *Monitoring & Maintenance → Establish Monitoring / Governance Operations*

C13 — Datastore choice not aligned with solution needs

- **Category:** Architecture / Data Platform
- **Description:** The solution uses spreadsheets or lists where a more suitable relational or mission-critical store should be used. The KB recommends leveraging Dataverse as the central data store for relational or mission-critical data where applicable. [\[Power Plat...Practices | Word\]](#)
- **Typical symptoms:** Performance issues under relational complexity, weak scalability, brittle query patterns.
- **Validation signals:** Heavy reliance on flat-file or list-based storage for relational or high-scale workloads.

- **Recommended actions:** Re-evaluate data platform choice, move appropriate workloads to Dataverse, redesign access patterns around the proper store. This is derived directly from the architecture guidance in the KB. [\[Power Plat...Practices | Word\]](#)
- **Default likelihood:** Medium
- **Default impact:** High
- **Correlation candidates:** Delegation limits, large dataset, monolithic design
- **KB reference:** *Architecture & Solution Design Principles → Use the Right Tool for the Job*

Performance Remediation Primitives

This section defines a set of **reusable remediation primitives** that guide how performance issues should be resolved across Power Platform solutions.

These primitives represent **common optimization strategies** that can be combined and adapted depending on the diagnosed issue.

These primitives are intended to be used by Copilot agents and solution architects to generate **consistent, scalable, and actionable remediation steps**.

List of Remediation Primitives

Reduce Data Volume

When to apply:

- Large datasets (e.g., SharePoint lists > 2,000 items)
- Slow queries or filtering operations
- Full dataset retrieval before filtering

What to do:

- Apply filters at the source before retrieving data
- Limit results to the records required for the current task
- Use indexed columns or pre-filtered views

Why it helps:

- Reduces data transferred to the client
- Improves load time and responsiveness

Push Processing to the Data Source

When to apply:

- Delegation warnings are present
- Filtering or logic runs client-side
- Performance degrades with dataset size

What to do:

- Use delegable functions (Filter, Lookup, Sort, StartsWith)

- Ensure filtering and sorting happen at the data source
- Avoid post-retrieval processing

Why it helps:

- Leverages backend execution
- Improves scalability and accuracy

Replace Non-Scalable Patterns

When to apply:

- Solution works with small datasets but not at scale
- Performance degrades as data grows

What to do:

- Remove full dataset ClearCollect() patterns
- Replace non-delegable formulas
- Avoid repeated full data reloads

Why it helps:

- Prevents exponential performance degradation

Defer Non-Essential Processing

When to apply:

- Slow app startup or screen load
- Heavy logic in OnStart or OnVisible

What to do:

- Move non-critical logic out of startup events

- Load data on-demand
- Trigger operations via user actions when appropriate

Why it helps:

- Improves perceived performance
- Reduces initial load time

Limit Data Payload

When to apply:

- Slow data retrieval or rendering
- Excessive fields, attachments, or images

What to do:

- Retrieve only required columns
- Avoid loading heavy fields unless needed
- Preserve Explicit Column Selection behavior
- Use ShowColumns() only when column lineage is lost

Why it helps:

- Reduces network overhead
- Improves rendering performance

Simplify Execution Logic

When to apply:

- Complex formulas or nested conditions
- Performance issues tied to computation

What to do:

- Break formulas into smaller reusable parts
- Remove repeated or nested calculations
- Use named formulas when appropriate

Why it helps:

- Reduces computation cost
- Improves maintainability

Control Execution Frequency

When to apply:

- Repeated refresh or recalculation
- Same logic executed unnecessarily

What to do:

- Avoid data reload on every screen navigation
- Trigger refresh only when needed
- Cache results when appropriate

Why it helps:

- Reduces redundant operations
- Improves responsiveness

Optimize UI Rendering

When to apply:

- UI responsiveness issues
- Heavy galleries or screens

What to do:

- Reduce number of controls rendered
- Avoid expensive formulas per item
- Load detail content only on user interaction

Why it helps:

- Improves rendering speed and usability

Isolate the Bottleneck

When to apply:

- Root cause is unclear
- Multiple factors may contribute to slowness

What to do:

- Identify the slowest component (formula, query, screen, flow step)
- Temporarily disable logic to isolate impact
- Use Monitor or telemetry tools for analysis

Why it helps:

- Enables accurate diagnosis before applying fixes

Restructure Solution Architecture

When to apply:

- Performance issues are systemic
- Local optimizations are insufficient

What to do:

- Split large apps or flows into modular components
- Separate UI logic from background processing
- Re-evaluate data platform (e.g., Dataverse vs SharePoint)

Why it helps:

- Enables long-term scalability and maintainability

Apply Configuration or Governance Fixes

When to apply:

- Performance is impacted by configuration or environment

What to do:

- Validate indexed columns and data structure
- Review connector configuration and environment setup
- Align permissions and governance policies with usage patterns

Why it helps:

- Eliminates hidden bottlenecks due to configuration issues

Validate Improvements with Metrics

When to apply:

- After implementing performance fixes

What to do:

- Compare before/after performance metrics

- Use Monitor or analytics tools
- Confirm removal of delegation warnings
- Validate response times and behavior improvements

Why it helps:

- Ensures changes are effective
- Prevents regressions

Common Primitive Combinations

These combinations are frequently effective for performance issues:

- **Reduce Data Volume + Push Processing to Source**

For delegation and large dataset issues

- **Defer Processing + Control Execution Frequency**

For startup and navigation slowness

- **Limit Payload + Optimize UI Rendering**

For slow screen rendering

- **Replace Non-Scalable Patterns + Restructure Architecture**

For systemic performance issues

Official Microsoft Learn References

Review these official Microsoft Learn articles for the latest ALM and governance guidance:

- [Application lifecycle management \(ALM\) with Microsoft Power Platform](#)
- [Solution concepts with Power Platform](#)

- [Overview of pipelines in Power Platform](#)
- [Microsoft Power Platform Build Tools for Azure DevOps](#)
- [Microsoft Power Platform CLI](#)
- [Security and governance considerations in Power Platform](#)
- [Power Platform Center of Excellence \(CoE\) Starter Kit transition to Power Platform admin center](#)

Conclusion

Following these best practices helps ensure that Power Platform solutions are robust, secure, performant, and manageable. By integrating these habits into the development lifecycle, teams reduce technical debt and give AI assistants and team members a strong, standardized foundation to work from. Always adapt best practices to the specific context of the project and balance the pillars of the Well-Architected Framework. The goal is a **well-rounded, enterprise-ready solution**.